SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA47 – DEEP LEARNING

ASSIGNMENT – 2 : SOLUTIONS

CO2 – Deep Feed-Forward and Convolutional Network Design

Name:	RENEE VINCENT ASR	Register No:	192424161
Course:	CSA4708 – Deep Learning	Total Marks:	50

Scenario: Smart agriculture. A research centre wants (i) a feed-forward network that predicts crop yield from rainfall, temperature, soil moisture and fertilizer quantity, and (ii) a convolutional network that classifies leaf images as healthy or diseased. All calculations below are worked out step by step so that they can be checked by hand.

Task 1 – Deep Feed-Forward Network Architecture

a) Network design

Yield prediction from four numeric measurements is a regression problem, so the network has one output neuron.

Layer	Neurons	Role / activation
Input	4	Rainfall, temperature, soil moisture, fertilizer (normalized); no computation
Hidden layer 1	8	Learns combinations of the inputs; ReLU
Hidden layer 2	4	Learns higher-level combinations; ReLU
Output	1	Predicted yield $\hat{y}$; linear (identity) activation, since yield is a real number

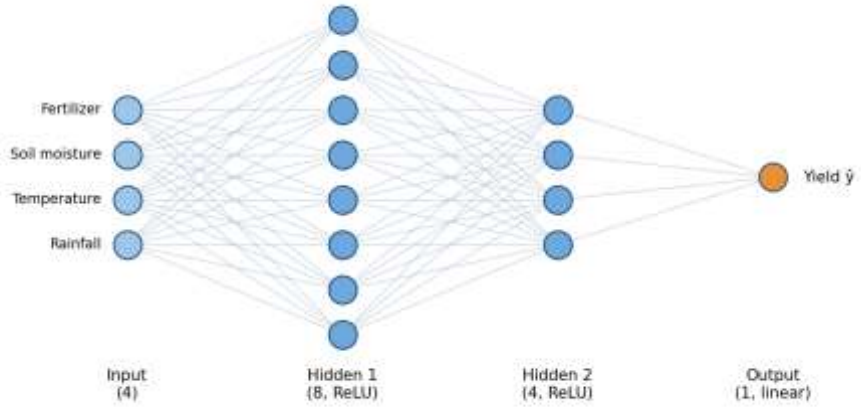


Figure 1: The 4-8-4-1 feed-forward network

b) Weight and bias dimensions

With the convention $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$, the weight matrix of layer l has shape (neurons in layer l) × (neurons in layer l−1).

Layer	Weight matrix	Bias vector	Parameters
Input → Hidden 1	$W^{(1)}$: 8 × 4	$b^{(1)}$: 8 × 1	32 + 8 = 40
Hidden 1 → Hidden 2	$W^{(2)}$: 4 × 8	$b^{(2)}$: 4 × 1	32 + 4 = 36
Hidden 2 → Output	$W^{(3)}$: 1 × 4	$b^{(3)}$: 1 × 1	4 + 1 = 5
Total			81

c) How inputs flow to the prediction (forward propagation)

The input vector x = [rainfall, temperature, moisture, fertilizer]^T is a 4 × 1 column. It is passed forward one layer at a time:

$$a^{(0)} = x$$

$$z^{(1)} = W^{(1)}x + b^{(1)} \ (8\times1), \quad a^{(1)} = ReLU(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \ (4\times1), \quad a^{(2)} = ReLU(z^{(2)})$$

$$\hat{y} = W^{(3)}a^{(2)} + b^{(3)} \ (1\times1)$$

Every neuron takes a weighted sum of the previous layer's outputs, adds its bias and applies the activation. Because ReLU is non-linear, stacking the layers lets the network model curved relationships (for example, extra fertilizer helps up to a point and then stops helping), which a single linear layer could not. The final number ŷ is the predicted crop yield.

Task 2 – Forward Propagation and Activation Functions

Inputs x = (0.6, 0.8, 0.5), weights w = (0.4, 0.3, 0.2), bias b = 0.1.

a) Weighted sum

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$z = (0.4)(0.6) + (0.3)(0.8) + (0.2)(0.5) + 0.1 = 0.24 + 0.24 + 0.10 + 0.10 = 0.68$$

b) ReLU and interpretation

$$a = ReLU(z) = max(0, 0.68) = 0.68$$

Since z is positive, ReLU passes it unchanged and the neuron outputs 0.68. It means this neuron is "active" for the present input: the combination of rainfall, temperature and moisture (as weighted by the neuron) is strong enough to send a signal to the next layer. Had z been negative, the output would be 0 and the neuron would stay silent for that input.

Task 3 – Gradient-Based Learning and Loss Calculation

a) Why a loss function is needed and which one for regression

A network starts with random weights, so its first predictions are poor. A loss function turns "how wrong is the prediction" into a single number, which gives the training algorithm something to minimise and a direction (the gradient) in which to move the weights. Without it, there is no measurable objective.

For regression, a suitable choice is the **mean squared error (MSE)**, $L = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$. It is smooth and differentiable, and it penalises large errors more than small ones. Mean absolute error (MAE) is an alternative that is less sensitive to outliers.

b) Squared error loss

$$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = 64$$

c) Interpretation

The model under-predicts yield by 8 units, and the loss of 64 quantifies this error. A loss of 0 would mean a perfect prediction. By itself, 64 has meaning only relative to the scale of the data and to earlier loss values: if it falls during training the model is improving. Squaring makes the penalty grow quickly (an error of 16 would give 256, four times larger, not twice).

The gradient of the loss with respect to the output is $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -2(8) = -16$. The negative sign says that increasing $\hat{y}$ would decrease the loss, which is exactly what the model needs to do.

d) How gradient-based learning uses the loss

- **Forward pass:** compute $\hat{y}$ and the loss L.
- **Gradient:** compute $\partial L/\partial \theta$ for each weight and bias θ using backpropagation. It tells how much and in which direction the loss changes when θ changes slightly.
- **Update:** move each parameter a small step against its gradient: $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$, where η is the learning rate.
- **Repeat** over many batches and epochs until the loss stops decreasing. The loss surface is followed downhill, like walking down a slope in fog by feeling the ground.

Task 4 – Gradient Descent and Weight Update

a) Updated weight

$$w_new = w - \eta\, (\partial L/\partial w) = 0.5 - (0.1)(0.8) = 0.5 - 0.08 = 0.42$$

b) Why the update opposes the gradient

The gradient points in the direction in which the loss increases most steeply. We want to reduce the loss, so we move the opposite way. Here the gradient is +0.8, meaning that increasing w would increase the loss, hence w is decreased from 0.5 to 0.42. If the gradient were negative, the same rule would increase w. For a small enough step the loss is guaranteed to go down (first-order Taylor approximation: $\Delta L \approx -\eta \|\nabla L\|^2$).

c) Effect of an extremely small or large learning rate

Learning rate	**Effect**	**Example with w = 0.5, gradient 0.8**
Extremely small (η = 0.0001)	Very slow convergence, needs huge numbers of iterations; may get stuck on plateaus or saddle points	w_new = 0.5 − 0.00008 = 0.49992 (almost no change)
Suitable (η = 0.1)	Steady decrease of loss	w_new = 0.42
Extremely large (η = 10)	Overshoots the minimum, loss oscillates or diverges	w_new = 0.5 − 8 = −7.5 (jumps far past the minimum)

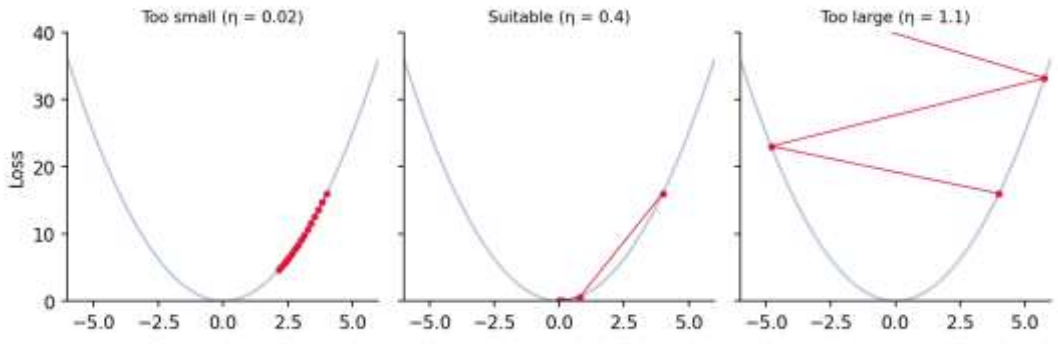


Figure 2: Gradient descent on $L = w^2$ with different learning rates

Task 5 – Back-Propagation Through Multiple Layers

a) How backpropagation finds each weight's contribution to the error

After the forward pass the network has a prediction $\hat{y}$ that differs a lot from the actual yield, so the loss L is large. Backpropagation then works backwards from the output layer to the input layer. At the output it computes the error signal $\delta^{(3)} = \partial L/\partial z^{(3)}$. This signal is sent back through the weights to give $\delta^{(2)}$, then $\delta^{(1)}$. The gradient of each weight matrix is then the product of the error signal of its layer and the activation coming into it:

$$\delta^{(l)} = \left(W^{(l+1)T} \delta^{(l+1)} \right) \odot f'\left(z^{(l)} \right), \quad \partial L/\partial W^{(l)} = \delta^{(l)} \left(a^{(l-1)} \right)^T, \quad \partial L/\partial b^{(l)} = \delta^{(l)}$$

A weight that strongly influenced the output (large activation, large downstream weights) receives a large gradient, i.e. a large share of the "blame" for the error.

b) Role of the chain rule

The loss depends on an early weight only through many nested functions: weight → z → activation → next layer … → $\hat{y}$ → L. The chain rule multiplies the local derivatives along this path. For a weight w in the first hidden layer:

$$\partial L/\partial w = (\partial L/\partial \hat{y}) \cdot (\partial \hat{y}/\partial a^{(2)}) \cdot (\partial a^{(2)}/\partial a^{(1)}) \cdot (\partial a^{(1)}/\partial z^{(1)}) \cdot (\partial z^{(1)}/\partial w)$$

Backprop reuses the products already computed for later layers, so all gradients are obtained in a single backward sweep with a cost close to that of the forward pass, instead of recomputing each from scratch.

Tiny numerical example (one neuron per layer: x = 1, $w_1 = 0.5$, $w_2 = 0.4$, ReLU hidden, y = 1):

- Forward: h = ReLU(0.5·1) = 0.5; $\hat{y} = 0.4 \cdot 0.5 = 0.2$; $L = (1 - 0.2)^2 = 0.64$
- $\partial L/\partial \hat{y} = -2(1 - 0.2) = -1.6$
- $\partial L/\partial w_2 = (-1.6)(h) = (-1.6)(0.5) = -0.80$
- $\partial L/\partial h = (-1.6)(w_2) = -0.64$; $\partial L/\partial w_1 = (-0.64)(\text{ReLU}' = 1)(x = 1) = -0.64$

c) Using the gradients to update the weights

With $\eta = 0.1$: $w_2 \leftarrow 0.4 - 0.1(-0.80) = 0.48$ and $w_1 \leftarrow 0.5 - 0.1(-0.64) = 0.564$. Re-running the forward pass gives $\hat{y} = 0.564 \times 0.48 = 0.271$ and $L = (0.729)^2 \approx 0.532$, which is lower than 0.64. The same update is applied to every weight and bias in the real network, and the process is repeated for each mini-batch.

Task 6 – Activation Functions and Vanishing/Exploding Gradients

a) How activation functions contribute to vanishing gradients

During backpropagation the gradient reaching an early layer is a product of many terms, and each term includes the derivative of the activation function. The sigmoid derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which is never above 0.25 and is almost 0 when $|z|$ is large (saturation). Multiplying many numbers below 0.25 shrinks the gradient exponentially: for a 10-layer network the activation factor alone is at most $0.25^{10} \approx 9.5 \times 10^{-7}$. The early layers then receive almost no learning signal and train very slowly, which is exactly the situation described. The opposite problem, **exploding gradients**, occurs when the multiplied terms (often the weights) are larger than 1, so gradients grow exponentially and the loss becomes unstable or NaN.

b) Comparison of sigmoid, tanh and ReLU for hidden layers

Property	**Sigmoid**	**tanh**	**ReLU**
Formula	$1/(1+e^{-z})$	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	$\max(0, z)$
Output range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$
Max derivative	0.25	1	1 (for $z > 0$)
Zero-centred	No	Yes	No
Saturation / vanishing	Severe	Present, milder	None for $z > 0$
Cost	Exponential, slower	Exponential, slower	Very cheap
Main issue	Vanishing gradient	Vanishing for large $\|z\|$	"Dying ReLU" (stuck at 0)
Typical use	Output for binary probability	Shallow nets, RNNs	Default for deep hidden layers

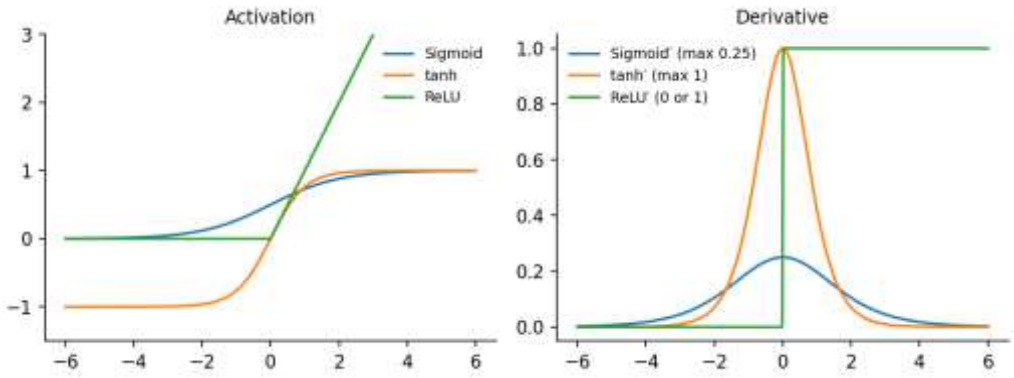


Figure 3: Activation functions and their derivatives

c) Effect of activation choice on training of deep networks

- ReLU keeps the gradient at exactly 1 for active neurons, so gradients travel back without shrinking, allowing deeper networks to be trained faster.
- Sigmoid and tanh in deep hidden stacks make learning in early layers slow; sigmoid is better kept for the output of binary classification.
- Dying ReLU can be reduced with Leaky ReLU or ELU and a small learning rate.
- Activation choice works together with weight initialisation (He initialisation for ReLU, Xavier for tanh), batch normalisation, residual connections and gradient clipping (against exploding gradients).

For the slow-learning crop network: replace sigmoid/tanh hidden units with ReLU (or Leaky ReLU), use He initialisation and add batch normalisation.

Task 7 – Back-Propagation and Optimization Strategy

a) Effect of the learning rate

The learning rate η sets the step size of each update. If it is too small, the loss decreases slowly (first experiment). If it is too large, the steps overshoot the minimum, and the loss falls quickly at the start but then bounces around or even rises (second experiment). A well-chosen η gives a fast and smooth decrease.

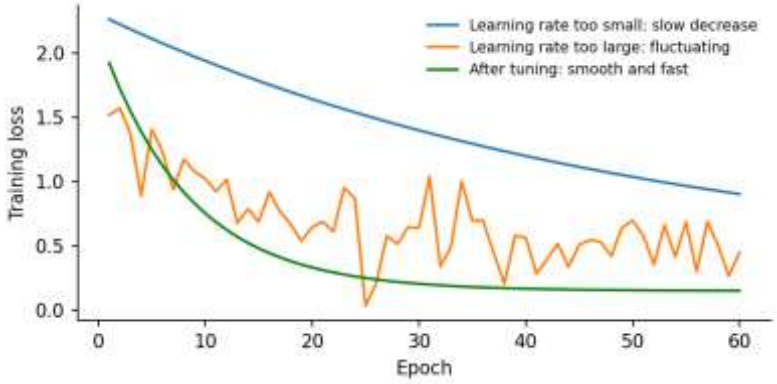


Figure 4: Typical loss curves for different learning rates

b) Backpropagation and gradient descent in one iteration

- Take a mini-batch and run the **forward pass** to get predictions.
- Compute the **loss** for the batch.
- Run **backpropagation** to get $\partial L/\partial W$ and $\partial L/\partial b$ for every layer (backprop only computes the gradients).
- Run **gradient descent**, $\theta \leftarrow \theta - \eta \nabla \theta L$, to change the parameters (the optimiser uses the gradients).

So backpropagation answers "which way and how much for each parameter?" and gradient descent answers "how far do we move?".

c) Improving convergence in both experiments

- **Slow decrease:** increase the learning rate (e.g. 10× steps in a learning-rate range test), use momentum or Adam, normalise inputs, use batch normalisation, check initialisation, and use a larger batch size if the gradients are too noisy.
- **Fluctuating loss:** reduce the learning rate, use a decay schedule (step, cosine or ReduceLROnPlateau), increase the batch size, add momentum/Adam, and use gradient clipping.

d) Why monitor the loss curve

The curve is the main diagnostic: slow decrease means η is too low, wild oscillation means η is too high, a plateau suggests a stuck optimisation or saturated units, a rise suggests divergence, and a growing gap between training and validation loss reveals overfitting. It also decides when to stop training (early stopping) and when to change the learning rate.

Task 8 – Regularization and Overfitting Control

a) The learning problem

The 99% training accuracy against 78% validation accuracy gives a generalization gap of 21 percentage points. The model has memorised the training data (including its noise) rather than learning patterns that transfer to new data, which means it is **overfitting** (high variance).

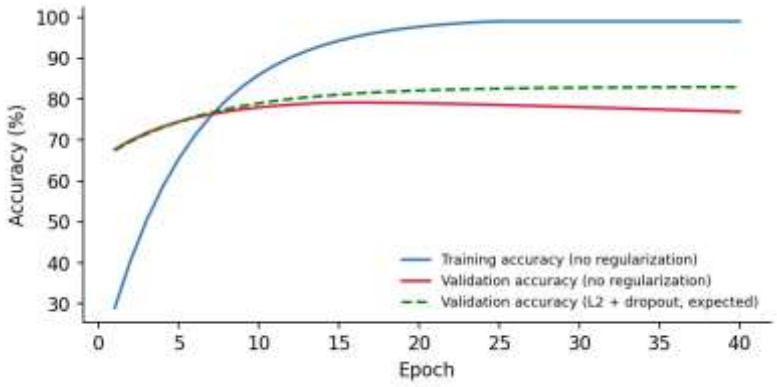


Figure 5: Overfitting and the expected effect of regularization

b) L1 and L2 regularization

- **L2 (weight decay)** adds $\lambda\Sigma w^2$ to the loss. Large weights are punished, so weights stay small and the function becomes smoother and less sensitive to noise. Weights shrink toward zero but rarely reach exactly zero.
- **L1** adds $\lambda\Sigma|w|$. It pushes many weights to exactly zero, giving sparse models and implicit feature selection.

Both limit the effective complexity of the network without removing neurons.

c) Regularized loss with an L2 penalty

$$L_reg = (1/n)\, \Sigma_i\, (y_i - \hat{y}_i)^2 \;+\; \lambda\, \Sigma_l \,\| W^{(l)} \|^2_F$$

The regularization parameter $\lambda \geq 0$ controls the trade-off between fitting the data and keeping weights small. $\lambda = 0$ means no regularization (overfitting risk); a very large λ forces weights close to 0 and the model underfits. A moderate λ (such as 10^{-4} to 10^{-2}) is picked by validation. In the gradient descent update it appears as weight decay: $w \leftarrow w(1 - 2\eta\lambda) - \eta\, \partial L_data/\partial w$. For example, with $w = 0.5$, $\eta = 0.1$ and $\lambda = 0.01$ the decay factor is 0.998, so w shrinks slightly at every step.

d) Dropout

During training, dropout randomly sets each hidden neuron's output to zero with probability p (for example 0.3 to 0.5), and a different random subnetwork is used for every mini-batch. Neurons cannot rely on specific other neurons (no co-adaptation), so features have to be robust and

redundant. It behaves like training and averaging a large ensemble of thinner networks. At test time all neurons are used, with outputs scaled (or inverted dropout scaling during training). Other helpful measures: early stopping, more data and data augmentation.

Task 9 – Convolution Operation and Feature Extraction

a) Why convolutional layers suit image data

- **Local connectivity:** spots, edges and texture on a leaf are local patterns spread over neighbouring pixels, and a filter looks at a small window.
- **Parameter sharing:** the same filter slides over the whole image, so a lesion is detected wherever it occurs (translation equivariance) with very few parameters.
- **Spatial structure is kept,** unlike a fully connected network, which flattens the image and loses neighbourhood relationships.
- **Hierarchy:** edges → textures → spots/lesion shapes → whole-leaf patterns.

b) Convolution example

A 5 × 5 grayscale patch (bright leaf area on the left, dark background on the right) is convolved with a 3 × 3 vertical-edge filter (stride 1, no padding). Output size = (5 − 3)/1 + 1 = 3, so the feature map is 3 × 3.

Image (5×5)				
9	9	9	0	0
9	9	9	0	0
9	9	9	0	0
9	9	9	0	0
9	9	9	0	0

Filter (3×3)		
1	0	−1
1	0	−1
1	0	−1

Top-left window (columns 1–3): each row gives 9(1) + 9(0) + 9(−1) = 0, so the output is 0. Next window (columns 2–4): each row gives 9(1) + 9(0) + 0(−1) = 9, so the output is 3 × 9 = 27. Last window (columns 3–5): 9(1) + 0 + 0 = 9 per row, so 27.

Feature map (3×3)		
0	27	27
0	27	27
0	27	27

Interpretation: the large values show where a bright-to-dark vertical edge exists. The first window is entirely uniform, so the response is 0. In a leaf image such a filter would respond strongly at the border of a disease spot or a leaf vein. The network learns the filter values during training instead of having them designed by hand.

c) Roles of filters, stride and padding

- **Filters (kernels):** small learnable weight grids, each one detecting a particular pattern (edge, spot, texture). Using F filters gives F feature maps.
- **Stride (S):** the number of pixels the filter moves each step. A larger stride gives a smaller feature map and cheaper computation but may skip details. With stride 2 the example above gives a 2 × 2 output ([0, 27; 0, 27]).
- **Padding (P):** zeros added around the border. It lets the filter see the edge pixels and can keep the output size the same ("same" padding; for a 5×5 input and a 3×3 filter with P = 1 the output stays 5×5).

$$\textit{Output size} = \lfloor (N + 2P - F) / S \rfloor + 1$$

Task 10 – CNN Architecture and Pooling

To make the dimensions concrete, assume a 64 × 64 RGB leaf image, two convolution layers (3×3, stride 1, padding 1) with 8 and 16 filters, 2×2 max pooling, and 2 output classes.

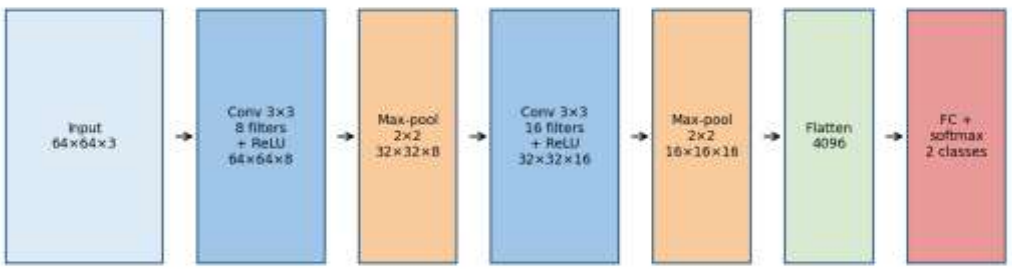


Figure 6: The proposed CNN with feature-map sizes

a) Function of each component

Component	Function
Input image	Raw pixels (64×64×3), normalized to 0–1
Convolution	Slides learnable filters over the image to produce feature maps that detect edges, spots and textures
ReLU	Adds non-linearity and keeps gradients healthy; negative responses become 0
Pooling	Down-samples each feature map; reduces size and computation and gives small shift tolerance
Second Conv + ReLU + Pool	Combines simple features into more complex ones (lesion shapes, patterns)
Fully connected layer	Flattens the final feature maps (4096 values) and combines them for classification
Output	Softmax over two classes: healthy / diseased

b) How pooling reduces dimensions

Pooling takes a small window (usually 2×2, stride 2) and replaces it by one number: the maximum (max pooling) or the average. This halves the height and width, so the map has only a quarter of the values. Example on a 4×4 feature map:

Input 4×4			
1	3	2	1
4	6	5	2
3	1	0	2
2	2	4	8

Max pooling gives [6, 5; 3, 8] and average pooling gives [3.5, 2.5; 2, 3.5]. Pooling has no learnable parameters. Output size = $\lfloor (N - F)/S \rfloor + 1 = (4 - 2)/2 + 1 = 2$.

Dimension tracking: 64×64×3 → conv (8 filters, pad 1) 64×64×8 → pool 32×32×8 → conv (16 filters) 32×32×16 → pool 16×16×16 → flatten 4096 → FC 2. Parameters: Conv1 = 3·3·3·8 + 8 = 224; Conv2 = 3·3·8·16 + 16 = 1,168; FC = 4096·2 + 2 = 8,194; total = 9,586.

c) Effect of filter size, stride and number of filters

- **Filter size:** small filters (3×3) capture fine detail with few parameters; larger filters (5×5, 7×7) see wider context but cost more and shrink the map more without padding.
- **Stride:** larger stride gives smaller feature maps, faster training and less detail.
- **Number of filters:** it sets the depth (channels) of the feature map. More filters learn more distinct patterns, but increase parameters, memory and overfitting risk.

d) Hierarchical feature learning

The first convolution layer sees raw pixels, so it learns simple features such as edges, colour blobs and gradients. The second layer works on these feature maps and combines them into textures, spots and vein patterns. Because pooling enlarges the area of the original image that each neuron "sees" (its receptive field), deeper layers respond to bigger and more abstract structures such as lesion shapes or a whole diseased region. The fully connected layer finally uses these high-level features to decide between healthy and diseased.

Task 11 – Comparative Evaluation of Feed-Forward and CNN Models

a) Why Model A suits structured numerical data

Weather and soil measurements form a small fixed-length vector of independent features with no spatial layout. A feed-forward network with fully connected layers can learn non-linear combinations of these variables (rain × temperature, fertilizer saturation) with few parameters and little computation, and it is fast to train and easy to analyse.

b) Why Model B suits images

A leaf image has thousands of pixels with strong local correlation, and disease signs (spots, edges, texture) can appear anywhere. Convolution and pooling exploit local structure, share weights and build hierarchical features. A fully connected network on a 64×64×3 image would need 12,288 weights for each neuron in its first layer, for only 8 neurons that is already 98,312 parameters, and it would not generalise across positions.

c) Comparison

Aspect	Model A (feed-forward)	Model B (CNN)
Input representation	Vector of 4 normalized numbers	3-D tensor (height × width × channels) of pixels

Feature extraction	Learned by hidden layers from the given features; features are chosen by experts	Learned automatically by convolution filters, from edges to complex shapes
Parameter organisation	Dense matrices, every neuron connected to every input (e.g. 81 parameters for 4-8-4-1)	Small shared filters reused at all positions, followed by a dense layer (about 9.6 thousand in the small example, millions in real CNNs)
Computation	Very low; trains on a CPU in seconds or minutes	High; convolutions on every position, needs a GPU, memory for feature maps
Data needed	Thousands of records	Thousands to hundreds of thousands of images (or transfer learning)
Output	Continuous yield (regression)	Class probabilities (classification)

d) Recommendation

- **Yield prediction → Model A.** The data are structured numbers, the problem is regression, and a small network is cheap, quick and less prone to overfitting.
- **Leaf disease classification → Model B.** The input is images, where spatial patterns decide the class, so a CNN is the natural and more accurate choice.

The two tasks use different data types, so each gets the architecture suited to it; they could later be combined (image features + weather data) to improve disease prediction.

Task 12 – Design Challenge: Optimize and Redesign the Deep Learning System

Decision: the existing system should be **redesigned**, with part of it retained. The feed-forward model remains valid as the starting point for the numerical yield task, but it needs better optimisation and regularisation (modify). Leaf images need a new, separate CNN branch (redesign). The overall system is therefore a two-branch design.

A. Impact Analysis

a) Effect of more data and the addition of images

- **Numerical data (10,000 → 500,000 records):** 50 times more data. With a mini-batch of 256, one epoch is about 1,954 iterations. Full-batch training is impractical, so mini-batch SGD/Adam with efficient data loading is required. More data lowers overfitting, so a somewhat larger network becomes affordable, but training time per epoch rises 50-fold.
- **Images (100,000 leaf images):** each 224×224×3 image is about 150 KB as 8-bit data, so the dataset is about 15 GB (about 60 GB if stored as float32). It will not fit in memory, so streaming input pipelines, on-the-fly augmentation, and a GPU are needed. Convolution is much more expensive than a dense layer, so FLOPs and memory for feature maps become the key constraints.
- **Architecture:** one network cannot sensibly handle both; the images require convolutional layers, while the numerical data keep a feed-forward design.

b) Limitations of a feed-forward network applied directly to images

- Flattening destroys the spatial neighbourhood, so edges and texture relationships are lost.
- A very large number of parameters: 224×224×3 = 150,528 inputs, so even 100 neurons in the first layer needs about 15 million weights, which invites severe overfitting and heavy memory use.
- No translation invariance: the same lesion in a different position looks like a different input and must be learnt again.
- Poor feature hierarchy and weak accuracy on visual tasks.

B. Evaluation of the Existing Design

a) Training vs validation performance. High training accuracy with lower validation performance shows a large generalization gap, so the existing model is overfitting (high variance). The long training time shows an inefficient optimisation or a model and pipeline too heavy for the data size (for example full-batch updates, no learning-rate schedule). The model is therefore not reliable on new fields or seasons.

b) Is increasing depth and neurons the best fix? No. A model that already overfits has too much capacity for what its data and regularisation can support, so adding capacity usually increases the gap and the cost. More depth also brings vanishing-gradient trouble and longer training. A better approach is to improve generalisation (L2, dropout, early stopping, batch normalisation, more and cleaner data), tune the optimiser and learning rate, and choose an architecture matched to the data type. Depth is increased only if validation error, not training error, is still high, which would indicate underfitting.

C. Adaptation or Redesign

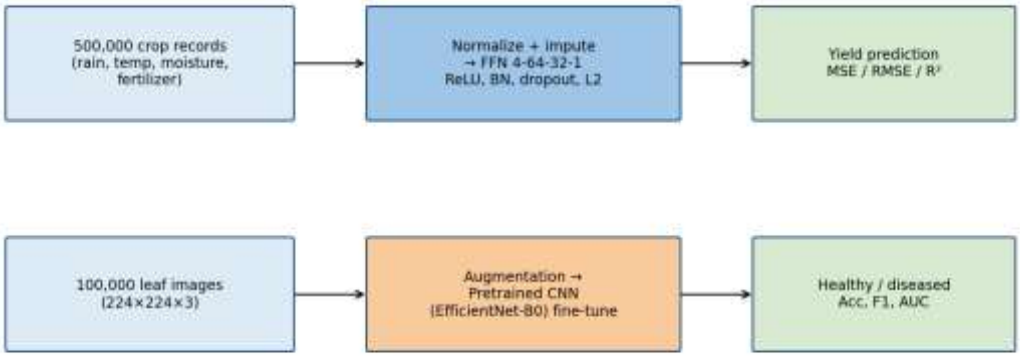


Figure 7: Redesigned two-branch system

a) Proposed architectures

- **Numerical branch (regression):** inputs normalized (z-score) with training statistics, missing values imputed; network 4 → 64 → 32 → 1 with ReLU hidden layers, batch normalisation, dropout (0.2), He initialisation, linear output, MSE (or Huber) loss. About 2,400 parameters, small enough for 500,000 records to train in minutes.
- **Image branch (classification):** a CNN using transfer learning. A pretrained EfficientNet-B0 (or ResNet-18/50) is fine-tuned with a new 2-class head; the first stage trains only the head, then the upper layers are unfrozen with a smaller learning rate. Input 224×224, normalized with the ImageNet mean and std; softmax (or sigmoid) with cross-entropy.

b) Optimization and regularization choices

Method	Recommendation	Reason
Optimiser	Adam (or AdamW) for both; SGD with momentum 0.9 as a strong alternative for CNN fine-tuning	Adaptive steps and fast, stable convergence
Learning-rate	Mini-batch size 256 (numerical), 64 (images); warm-up for 3–5 epochs, then cosine decay; or ReduceLROnPlateau	Quick start, smooth finish, avoids oscillation
Dropout	0.2 in the dense branch, 0.3–0.5 before the CNN classifier head	Reduces co-adaptation
L2 / weight decay	$\lambda \approx 10^{-4}$ (AdamW weight decay 0.01)	Keeps weights small
Batch normalisation	After each conv/dense layer, before activation	Stable gradients, allows higher learning rate

Activation	ReLU (hidden), linear output for yield, softmax/sigmoid for disease	No saturation in hidden layers
Early stopping	Patience of 5–10 epochs on validation loss	Stops before overfitting

D. Evidence-Based Defence

a) Experiment design

Split the data into 70% training, 15% validation and 15% test, with a stratified split for the leaf classes and a split by field/season where possible to avoid leakage. Use the same splits for every model. Run each configuration with 3 random seeds, and report mean ± standard deviation.

System	Task	Metrics
Original FFN (baseline)	Yield and images (flattened)	Train/val loss, RMSE, MAE, R^2, accuracy, F1, training time
Redesigned FFN (4-64-32-1)	Yield regression	Training and validation MSE, RMSE, MAE, R^2, time per epoch, total time
Redesigned CNN (EfficientNet-B0)	Leaf classification	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix, training time

Compare train–validation gaps, learning curves, and wall-clock time to reach the best validation score. Ablation runs remove one change at a time (dropout, L2, batch norm, augmentation, scheduler) to show which helps. Use confidence intervals or a paired test to check that differences are real.

b) Justification

The redesigned system is expected to show a smaller generalisation gap, better validation RMSE/F1, and shorter training time per useful epoch, because regularisation controls variance, Adam with scheduling speeds up convergence, and a CNN uses the spatial structure of images with far fewer parameters than a dense network. If the experiments confirm this, the original model should be **modified** for the numerical task (kept as the base but regularised and optimised) and **replaced** by a CNN for image data. If the new numerical model gave no gain, the simpler original could be retained.

Additional design considerations

1. Comparison of optimizers

Optimiser	How it works	Strength	Weakness
Batch GD	Gradient on the whole dataset	Exact, smooth path	Very slow and memory-heavy for 500,000 records
SGD (batch = 1)	One example per update	Fast updates, can escape shallow minima	Very noisy
Mini-batch SGD	Small batch (32–256) per update	Good balance, GPU-efficient	Needs a tuned learning rate, slow in ravines
Momentum	Adds a velocity term: $v = \beta v + \nabla L$	Smooths noise, faster in ravines	Extra hyperparameter, can overshoot
RMSProp	Divides step by running average of squared gradients	Adapts per-parameter rate, good for non-stationary problems	No momentum, sensitive to η
Adam	Momentum + RMSProp with bias correction	Fast, robust defaults ($\eta = 10^{-3}$)	Can generalise slightly worse than tuned SGD

2. Learning-rate schedule: linear warm-up from 10^{-5} to 10^{-3} over the first 3 epochs, then cosine decay to 10^{-6} by epoch 30; when fine-tuning pretrained layers, use about 10 times smaller learning rate (10^{-4}) than for the new head. Alternatively, halve the rate whenever validation loss has not improved for 3 epochs.

3. Label-preserving image augmentation: the disease label must not change after the transformation.

- Use: horizontal and vertical flips, rotations up to 360° (leaves have no fixed orientation), small zoom or random-resized crops (scale 0.8–1.0), small brightness and contrast changes for varying sunlight, slight Gaussian blur or noise for camera quality.
- Avoid: strong colour or hue shifts (yellowing or browning is a symptom), very aggressive crops that could cut out the lesion, heavy occlusion or cut-and-paste of patches, and elastic distortions that change the lesion shape.
- Augment only the training set and keep validation and test images unchanged.

4. Comparison of image architectures (approximate published figures for ImageNet-style models)

Model	Typical input	Parameters	Compute	Remarks for this project

LeNet-5	32×32 gray	≈ 60 thousand	Tiny	Too small for detailed 224×224 colour leaf textures; fine for a quick baseline
AlexNet	227×227	≈ 61 million	≈ 0.7 GFLOPs	Big dense layers, heavy overfitting, outdated
ResNet-50	224×224	≈ 25.6 million	≈ 4 GFLOPs	Residual connections solve vanishing gradients; strong, but heavier (ResNet-18: 11.7 M parameters)
EfficientNet-B0	224×224	≈ 5.3 million	≈ 0.4 GFLOPs	Best accuracy per FLOP; small, suits limited compute

Choice: EfficientNet-B0 with pretrained weights, fine-tuned on the 100,000 images (ResNet-18/50 as the alternative or comparison baseline). It gives high accuracy for 224×224 images with low parameter count and compute, so it trains on a single GPU in reasonable time. Pretrained features also reduce the number of epochs and the overfitting risk.

Conclusion: the evidence-based plan is to modify the feed-forward model for yield prediction, add a transfer-learning CNN for leaf images, tune optimisation and regularisation, and accept the redesign only if the experiments show better validation performance and acceptable training time.

Summary of Tasks and Marks

Task	Topic	Marks
1	Deep feed-forward network architecture	5
2	Forward propagation and activation functions	2
3	Gradient-based learning and loss calculation	6
4	Gradient descent and weight update	4
5	Back-propagation through multiple layers	3
6	Activation functions and vanishing/exploding gradients	3
7	Back-propagation and optimization strategy	4
8	Regularization and overfitting control	4
9	Convolution operation and feature extraction	3
10	CNN architecture and pooling	4
11	Comparison of feed-forward and CNN models	4

12	Design challenge	8
	Total	50